

Sistemas Embebidos

Material de la materia

UADE

UNIVERSIDAD ARGENTINA

TP5: Protocolo serie con framing y checksum sobre UART en STM32F103

Dr. Ing. Leonardo J. Amet

Referencia de bloque: trabajo práctico sobre protocolo serie UART con tramas, checksum XOR y parser incremental en Blue Pill. El firmware implementa el mismo protocolo que usa el proyecto **bluepill-freertos-ros2** de la cátedra. El bridge ROS 2 lo provee la cátedra como arnés de prueba: el estudiante no tiene que desarrollarlo ni modificarlo.

1 Objetivos del trabajo práctico

Al terminar este TP, el estudiante debería poder:

- Ubicar y entender la configuración de USART1 en el esqueleto firmware de referencia de la cátedra: PA9 (TX), PA10 (RX), 115200 baud, 8N1, con libopenm3.
- Rastrear el flujo de transmisión y recepción UART en el firmware de referencia, entendiendo los flags TXE, TC y RXNE del registro SR.
- Construir tramas del protocolo `@LL:TTT:PAYLOAD:CC\n` calculando LL (longitud hexadecimal de TTT:PAYLOAD) y CC (checksum XOR sobre LL:TTT:PAYLOAD).
- Implementar un parser incremental (FSM) que consuma bytes de UART y entregue tramas completas con checksum validado.
- Despachar los comandos `ping`, `led=on`, `led=off`, `led=toggle` y `status?`, respondiendo con tramas `ACK`, `STS` o `ERR` según corresponda.
- Diagnosticar fallas típicas del protocolo: checksum inválido, framing roto, comandos desconocidos y resincronización tras ruido en la línea.

2 Introducción

Hasta ahora mandaste y recibiste bytes por UART como caracteres sueltos. Pero en un sistema real los bytes no viajan solos: viajan en mensajes. Para que el receptor entienda dónde empieza un mensaje, qué tipo de mensaje es, cuánto mide y si llegó sin errores, necesitás un protocolo.

En este TP vas a implementar el mismo protocolo serie que usa el proyecto **bluepill-freertos-ros2** de la cátedra. El formato de trama es:

`@LL:TTT:PAYLOAD:CC\n`

Cada campo cumple una función precisa. `@` marca el inicio de la trama. LL son dos dígitos hexadecimales que indican la longitud del cuerpo `TTT:PAYLOAD`. TTT es el tipo de mensaje en tres letras mayúsculas: `CMD` para comandos, `ACK` para confirmaciones, `STS` para estado y `ERR` para errores. `PAYLOAD` es el contenido del mensaje en ASCII imprimible. `CC` es el checksum XOR de 8 bits, en hexadecimal, calculado sobre `LL:TTT:PAYLOAD`. `\n` cierra la trama.

La cátedra te da un bridge ROS 2 ya construido que se conecta al puerto serie de tu computadora. El bridge envuelve tus comandos en tramas del protocolo, las manda por UART a la Blue Pill, y publica las respuestas en tópicos de ROS 2 (`bridge/ack`, `bridge/status`, `bridge/error`). No tenés que tocar el bridge ni escribir una línea de ROS 2. Tu trabajo es que el firmware de la Blue Pill entienda las tramas que recibe, ejecute los comandos y responda con tramas bien formadas.

El TP avanza en tres etapas que construyen progresivamente la pila del protocolo. En la primera implementás la codificación y decodificación de tramas con checksum. En la segunda agregás un parser incremental que procesa los bytes que llegan por UART y despacha comandos de LED. En la tercera completás el manejo de errores, el comando de estado y la resincronización ante ruido.

3 Material de estudio obligatorio

Antes de arrancar el TP, tenés que leer:

- **Protocolo serie ROS 2 / Blue Pill** (apunte 13): formato de trama, checksum XOR, parser incremental, FSM y diagnóstico. Es el material central de este TP. Ahí se explica en detalle cada campo de la trama, la máquina de estados del parser y los modos de falla del protocolo.
- **Periféricos STM32F103**: sección USART del apunte 08 (registros BRR/CR1/SR/DR, transmisión por polling, checklist de diagnóstico). En particular, revisá las tablas de flags (TXE, TC, RXNE, ORE) y la sección “USART: más que baudios”.
- **Startup, linker y arranque** (apunte 01): para repasar la inicialización del clock y entender de dónde sale PCLK2.
- **Entorno y toolchain** (apunte 00): si necesitás refrescar cómo flasheás la Blue Pill y cómo te conectás por UART desde la computadora.

Material técnico complementario:

- Reference Manual RM0008 de ST, capítulo 27 (USART): para consultar los registros en detalle y los modos de operación.
- Datasheet del STM32F103C8T6: para confirmar la asignación de pines (PA9 = USART1_TX, PA10 = USART1_RX).
- Documentación del protocolo en el repo `bluepill-freertos-ros2` (<https://github.com/LeonardoAmet/bluepill-freertos-ros2>) con la especificación completa.

4 Base de trabajo

El TP se resuelve sobre el esqueleto firmware de referencia `bluepill-freertos-ros2` que provee la cátedra. Ese esqueleto ya incluye:

- USART1 configurado (PA9/PA10, 115200 8N1) con recepción por interrupción y cola de bytes.
- Clock a 72 MHz (HSE 8 MHz + PLL $\times 9$, PCLK2 = 72 MHz).
- GPIO con el LED de la placa en PC13 listo para usar.
- Infraestructura de FreeRTOS con las tareas de recepción y envío UART andando.

Tu trabajo es agregar las piezas del protocolo — codificación de tramas, checksum XOR, parser incremental y despacho de comandos — sobre esa base, sin modificar la arquitectura de recepción UART.

El bridge ROS 2 lo corre la cátedra en una computadora del laboratorio. Para desarrollo y prueba en tu casa, podés usar cualquier monitor serie (minicom, screen, PuTTY) que envíe y reciba las tramas en crudo. La sección de diagnóstico del apunte 13 explica cómo probar el protocolo sin el bridge.

Componentes necesarios:

- Blue Pill (STM32F103C8T6) con alimentación.
- Adaptador USB-UART (CP2102, CH340G, FT232 o similar).
- Cables dupont hembra-hembra para conectar TX/RX entre la Blue Pill y el adaptador.
- Computadora con monitor serie a 115200 baud, 8N1, sin control de flujo.

Conexionado mínimo:

Blue Pill	Adaptador USB-UART
PA9 (TX)	RX del adaptador
PA10 (RX)	TX del adaptador
GND	GND

El LED es el que ya está en la placa, conectado a PC13. No hace falta cablear nada adicional para el LED.

5 Consigna general

El TP se divide en tres etapas que construyen progresivamente un firmware capaz de comunicarse usando el protocolo `@LL:TTT:PAYLOAD:CC\n`:

1. **Framing y checksum:** implementar las funciones de codificación y decodificación de tramas. Dados un tipo y un payload, construir la trama completa calculando LL y CC. Dada una trama recibida, verificar el checksum y extraer tipo y payload.
2. **Parser incremental y comandos LED:** implementar la máquina de estados (FSM) que consume un byte por vez desde UART y entrega tramas completas y validadas. Despachar los comandos `ping`, `led=on`, `led=off` y `led=toggle`, respondiendo con tramas `ACK` y controlando el LED de la placa (PC13).
3. **Estado y errores:** agregar el comando `status?` que responde con una trama `STS` con los contadores internos. Manejar comandos desconocidos respondiendo con tramas `ERR`. Implementar la resincronización del parser ante ruido o tramas corruptas.

Cada etapa se apoya en la anterior. Si el framing y el checksum no funcionan, la FSM del parser no tiene sentido. Si el parser no discrimina tipos de mensaje, los comandos no se pueden despachar.

5.1 Comandos y respuestas esperadas

Tu firmware debe reconocer los siguientes comandos (tipo `CMD`) y responder según esta tabla:

Comando (payload)	Acción sobre la Blue Pill	Tipo de respuesta	Payload de respuesta
<code>ping</code>	Ninguna (solo confirmar enlace)	<code>ACK</code>	<code>pong=1</code>
<code>led=on</code>	Encender LED (PC13 en bajo)	<code>ACK</code>	<code>cmd=ok</code>
<code>led=off</code>	Apagar LED (PC13 en alto)	<code>ACK</code>	<code>cmd=ok</code>
<code>led=toggle</code>	Conmutar LED	<code>ACK</code>	<code>cmd=ok</code>
<code>status?</code>	Ninguna (reportar estado)	<code>STS</code>	<code>rx=N,ae=N,irq=N,pb=N,pm=N,pe=N,qd=N</code>
cualquier otro	Ninguna	<code>ERR</code>	<code>code=unknown_cmd</code>

Si la Blue Pill recibe una trama con tipo distinto de `CMD`, debe responder con `ERR` y payload `code=unexpected_type`.

5.2 Ejemplos de tramas

Para que tengas una referencia concreta, estos son los pares de trama enviada y trama esperada para cada comando. El checksum (CC) lo calcula el emisor con XOR byte a byte sobre `LL:TTT:PAYLOAD`. La verificación detallada de cada ejemplo es parte de la etapa 1.

```

ping:      TX @08:CMD:ping:52      → RX @0A:ACK:pong=1:22
led=on:    TX @0A:CMD:led=on:6A    → RX @0A:ACK:cmd=ok:6B
led=off:   TX @0B:CMD:led=off:07   → RX @0A:ACK:cmd=ok:6B
led=toggle: TX @0E:CMD:led=toggle:7D → RX @0A:ACK:cmd=ok:6B
status?:   TX @0B:CMD:status?:13   → RX @XX:STS:rx=N,ae=N,...:YY
desconocido: TX @XX:CMD:loquesea:YY → RX @14:ERR:code=unknown_cmd:2D

```

En la respuesta de `status?`, la longitud (XX) y el checksum (YY) dependen de los valores concretos de los contadores. El formato del payload siempre es `rx=N,ae=N,irq=N,pb=N,pm=N,pe=N,qd=N`.

5.3 Sobre el bridge ROS 2

El bridge `bluepill-freertos-ros2` que corre en la computadora del laboratorio se encarga de:

- Recibir comandos por tópicos de ROS 2, empaquetarlos en tramas del protocolo y enviarlos por UART a la Blue Pill.
- Recibir tramas de la Blue Pill, validarlas y publicar el tipo y payload en el tópico que corresponda.

Para este TP, el bridge es solo un arnés de prueba. No necesitás entender su implementación interna. Alcanza con saber que si tu firmware responde con tramas bien formadas, el bridge las reconoce y publica correctamente. Si tus tramas tienen errores de formato o checksum, el bridge las descarta, igual que haría cualquier implementación correcta del protocolo.

Importante: este TP no empieza desde cero. Trabajás sobre el esqueleto firmware `bluepill-freertos-ros2` que te da la cátedra. Ese esqueleto ya tiene la recepción UART por interrupción, la cola de bytes y las tareas de FreeRTOS andando. Te concentrás en agregar las piezas del protocolo: codificación de tramas, checksum, parser incremental y despacho de comandos.

6 Etapa 1: Framing y checksum

6.1 Consigna

En esta etapa implementás las funciones que codifican tramas del protocolo y validan el checksum de las tramas recibidas. No necesitás la UART todavía: podés probar las funciones con strings en memoria y verificar los resultados contra los ejemplos de la sección “Ejemplos de tramas” (arriba, en la consigna general).

El formato de toda trama es `@LL:TTT:PAYLOAD:CC\n`. En el apunte 13 encontrás la explicación completa de cada campo, las reglas de caracteres permitidos y los límites de tamaño (PAYLOAD máximo 48 caracteres, trama máxima 64 caracteres). Ahora te concentrás en dos funciones:

1. `protocol_encode(type, payload, buf, size)`: recibe un tipo ("CMD", "ACK", "STS" o "ERR") y un payload, construye la trama completa y la escribe en `buf`. Devuelve la cantidad de caracteres escritos (sin contar `\0` final) o `-1` si el buffer es muy chico.
2. `protocol_validate(frame, frame_len)`: recibe una trama SIN el `@` inicial ni el `\n` final (es decir, `LL:TTT:PAYLOAD:CC`). Recalcula el checksum XOR sobre `LL:TTT:PAYLOAD` y lo compara con `CC`. Devuelve 0 si es válido, `-1` si no.

Estas dos funciones son la base de todo lo que sigue. Si tus tramas tienen el checksum mal calculado, el parser de la etapa 2 nunca va a entregar mensajes válidos, y el bridge ROS 2 no va a reconocer tus respuestas.

6.2 Subtareas

6.2.1. 1.1 Calcular LL correctamente

LL mide solamente el cuerpo `TTT:PAYLOAD`. No cuenta `@`, ni los dos puntos externos, ni `CC`, ni `\n`.

Escribí una función auxiliar para convertir dígitos hexadecimales, que vas a necesitar tanto ahora como en el parser de la etapa 2:

```
static int hex_char_to_nibble(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1; // no es hex válido
}
```

Acepta solo mayúsculas A-F, igual que el protocolo. El parser rechaza minúsculas.

Verificá estos casos:

- CMD:ping → 8 caracteres → LL = "08"
- CMD:led=toggle → 14 caracteres → LL = "0E"
- ACK:cmd=ok → 10 caracteres → LL = "0A"
- ERR:code=unknown_cmd → 20 caracteres → LL = "14"

El formato de LL siempre es dos dígitos hexadecimales en mayúsculas, con cero a la izquierda si es menor que 0x10.

6.2.2. 1.2 Implementar el checksum XOR

El checksum es un XOR byte a byte sobre la cadena LL:TTT:PAYLOAD. No incluye @, el separador previo al checksum, CC ni \n.

```
uint8_t protocol_checksum(const char *input, size_t len) {
    uint8_t cs = 0U;
    for (size_t i = 0U; i < len; i++) {
        cs ^= (uint8_t)input[i];
    }
    return cs;
}
```

Para 08:CMD:ping, el resultado es 0x52. Podés verificarlo a mano siguiendo la tabla de acumulación byte a byte del apunte 13 (sección “Qué protege el checksum”), o escribiendo un test unitario en C.

El checksum siempre se formatea como dos dígitos hexadecimales en mayúsculas (`printf("%02X", cs)`). No uses `%02x` porque las minúsculas hacen fallar al parser.

6.2.3. 1.3 Implementar protocol_encode()

La función construye la trama completa siguiendo el procedimiento que describe el apunte 13 (sección “Generación de una trama”):

1. Construir el cuerpo TTT:PAYLOAD.
2. Medir la longitud del cuerpo y formatearla como LL (2 dígitos hex).
3. Construir LL:TTT:PAYLOAD.
4. Calcular checksum XOR sobre esa cadena.
5. Emitir @LL:TTT:PAYLOAD:CC\n.

```
int protocol_encode(const char *type, const char *payload,
                   char *buf, size_t buf_size)
{
    size_t type_len = strlen(type);
    size_t payload_len = strlen(payload);

    // Validar que el tipo tenga exactamente 3 letras
    if (type_len != 3) return -1;

    // Validar payload: solo ASCII imprimible, sin '@', máximo 48
    if (payload_len > PROTOCOL_MAX_PAYLOAD_LENGTH) return -1;
    for (size_t i = 0; i < payload_len; i++) {
        if (payload[i] < 0x20 || payload[i] > 0x7E || payload[i] == '@')
            return -1;
    }

    // 1. Cuerpo: TTT:PAYLOAD
    char body[PROTOCOL_MAX_BODY_SIZE + 1];
    snprintf(body, sizeof(body), "%s:%s", type, payload);
```

```

size_t body_len = strlen(body);

// 2-3. LL:TTT:PAYLOAD → checksum input
char cs_input[PROTOCOL_MAX_FRAME_LENGTH + 1];
int cs_bytes = snprintf(cs_input, sizeof(cs_input),
                        "%02X:%s", (unsigned int)body_len, body);
if (cs_bytes < 0 || (size_t)cs_bytes >= sizeof(cs_input)) return -1;

// 4. Checksum
uint8_t cs = protocol_checksum(cs_input, (size_t)cs_bytes);

// 5. Trama completa
int written = snprintf(buf, buf_size, "@s:%02X\n", cs_input, cs);
if (written < 0 || (size_t)written >= buf_size) return -1;

return written;
}

```

Las constantes `PROTOCOL_MAX_PAYLOAD_LENGTH` (48), `PROTOCOL_MAX_BODY_SIZE` (52 = 3+1+48) y `PROTOCOL_MAX_FRAME_LENGTH` (64) ya están definidas en `firmware/config/app_config.h` del esqueleto que te da la cátedra.

Verificá con estos casos:

- `protocol_encode("CMD", "ping", buf, 64)` → `buf` debe contener exactamente `@08:CMD:ping:52\n`.
- `protocol_encode("ACK", "cmd=ok", buf, 64)` → `buf` debe contener `@0A:ACK:cmd=ok:6B\n`.
- `protocol_encode("CMD", "led=toggle", buf, 64)` → `buf` debe contener `@0E:CMD:led=toggle:7D\n`.
- `protocol_encode("ERR", "code=unknown_cmd", buf, 64)` → `buf` debe contener `@14:ERR:code=unknown_cmd:2D\n`.

Si tu resultado no coincide con el de la tabla de ejemplos de la consigna general, revisá el algoritmo de checksum antes de seguir. Un solo byte mal calculado te va a romper todo el TP.

6.2.4. 1.4 Implementar `protocol_validate()`

Esta función supone que el parser ya extrajo la trama entre `@` y `\n`. Recibe `LL:TTT:PAYLOAD:CC` y solo verifica que el checksum coincida.

```

int protocol_validate(const char *frame, size_t frame_len) {
    // Buscar el ÚLTIMO ':' (separa checksum del resto)
    const char *last_colon = NULL;
    for (size_t i = 0; i < frame_len; i++) {
        if (frame[i] == ':') last_colon = &frame[i];
    }
    if (last_colon == NULL) return -1;

    // El checksum recibido debe ocupar exactamente 2 caracteres
    // al final del buffer: último ':' + 2 dígitos hex
    if (last_colon + 3 != frame + frame_len) return -1;
    const char *cs_received_str = last_colon + 1;

    // Convertir checksum recibido de hex string a uint8_t
    int hi = hex_char_to_nibble(cs_received_str[0]);
    int lo = hex_char_to_nibble(cs_received_str[1]);
    if (hi < 0 || lo < 0) return -1;
    uint8_t cs_received = (uint8_t)((hi << 4) | lo);
}

```

```

// El input para el checksum es todo lo anterior a ese ':'
size_t cs_input_len = (size_t)(last_colon - frame);

// Recalcular
uint8_t cs_calculated = protocol_checksum(frame, cs_input_len);

return (cs_calculated == cs_received) ? 0 : -1;
}

```

Probá estos casos:

- `protocol_validate("08:CMD:ping:52", 14)` → devuelve 0 (válido).
- `protocol_validate("08:CMD:ping:53", 14)` → devuelve -1 (checksum falso).
- `protocol_validate("0A:ACK:cmd=ok:6B", 16)` → devuelve 0.
- `protocol_validate("14:ERR:code=unknown_cmd:2D", 26)` → devuelve 0.

6.3 Preguntas obligatorias de la etapa 1

1. ¿Por qué LL cuenta solamente TTT:PAYLOAD y no el resto de la trama? ¿Qué problema práctico resolvería incluir los campos externos (@, los dos puntos que rodean al checksum, CC, \n) en la longitud?
2. Un estudiante calcula el checksum sobre @LL:TTT:PAYLOAD (incluyendo la arroba). ¿El checksum que obtiene es necesariamente distinto? ¿Funcionaría con el bridge ROS 2 de la cátedra? Justificá con el valor concreto para @08:CMD:ping.
3. La función `protocol_validate()` busca el *último* : de la trama para separar el checksum. ¿Podría buscar el *primer* :? ¿En qué situación concreta la elección del último es importante para que la validación funcione correctamente?

6.4 Evidencia mínima

- Código completo de `protocol_encode()` y `protocol_validate()`, incluyendo las funciones auxiliares `protocol_checksum()` y `hex_char_to_nibble()`.
- Salida de `protocol_encode()` para los cuatro comandos `ping`, `led=on`, `led=toggle` y `status?`, comparada con los valores de la tabla de ejemplos de la consigna general (no alcanza con copiarlos: tenés que mostrar que tu función los genera).
- Salida de `protocol_validate()` para al menos una trama válida y una con checksum alterado, mostrando que discrimina correctamente.

7 Etapa 2: Parser incremental y comandos LED

7.1 Consigna

En la etapa 1 construiste funciones para codificar y validar tramas. Ahora necesitás una pieza que consuma bytes crudos de UART y decida cuándo hay una trama completa. Esa pieza es el parser incremental.

Implementá un parser basado en una máquina de estados finitos (FSM) que consuma un byte por vez. Cuando complete una trama válida, el parser debe entregarte el tipo (CMD, ACK, ERR, etc.) y el payload. Con eso, tu firmware debe despachar los comandos `ping`, `led=on`, `led=off` y `led=toggle`, respondiendo con tramas ACK y controlando el LED de la placa (PC13).

El esqueleto `bluepill-freertos-ros2` que te da la cátedra ya tiene la recepción UART por interrupción y una cola de bytes. Los bytes que llegan del puerto serie se encolan en una FreeRTOS queue llamada `uart_rx_queue`. Tu parser va a consumir de esa cola, byte por byte, en una tarea dedicada.

7.2 Subtareas

7.2.1. 2.1 La máquina de estados del parser

El parser recorre 9 estados. Cada estado espera un tipo específico de byte. El diagrama completo está en el apunte 13 (sección “Máquina de estados del parser”). Los estados son:

#	Estado	Qué espera	Qué hace
0	WAIT_START	@ (0x40)	Ignora todo hasta encontrar el inicio de trama
1	READ_LEN_HI	Dígito hex	Guarda primer dígito de LL
2	READ_LEN_LO	Dígito hex	Guarda segundo dígito de LL, calcula longitud
3	EXPECT_LEN_SEPARATOR :		Confirma que LL terminó
4	READ_BODY	Bytes del cuerpo	Acumula TTT:PAYLOAD, cuenta cuántos bytes leyó
5	EXPECT_CHECK_SEPARATOR		Confirma que el cuerpo terminó
6	READ_CHECK_HI	Dígito hex	Guarda primer dígito de CC
7	READ_CHECK_LO	Dígito hex	Guarda segundo dígito de CC
8	EXPECT_END	\n (0x0A)	Valida checksum y entrega mensaje

El byte \r (0x0D) debe ignorarse en todo momento. No produce avance de estado ni error.

Definí el enum de estados y la estructura del parser:

```
typedef enum {
    PARSER_STATE_WAIT_START,
    PARSER_STATE_READ_LEN_HI,
    PARSER_STATE_READ_LEN_LO,
    PARSER_STATE_EXPECT_LEN_SEPARATOR,
    PARSER_STATE_READ_BODY,
    PARSER_STATE_EXPECT_CHECK_SEPARATOR,
    PARSER_STATE_READ_CHECK_HI,
    PARSER_STATE_READ_CHECK_LO,
    PARSER_STATE_EXPECT_END
} parser_state_t;

typedef enum {
    PARSER_RESULT_IN_PROGRESS,
    PARSER_RESULT_MESSAGE_READY,
    PARSER_RESULT_ERROR
} parser_result_t;

typedef struct {
    parser_state_t state;
    uint8_t body_len;           // longitud decodificada de LL
}
```

```

uint8_t body_pos;           // cuántos bytes del cuerpo ya se leyeron
char body_buf[PROTOCOL_MAX_BODY_SIZE + 1]; // buffer para TTT:PAYLOAD
uint8_t check_expected;     // checksum recibido (decodificado)
uint8_t check_hi_nibble;    // nibble alto, para armar CC
} parser_t;

```

Cada vez que llega un byte, llamás a la función del parser:

```

parser_result_t parser_consume_byte(parser_t *p, uint8_t byte,
                                   protocol_message_t *msg);

```

7.2.2. 2.2 Reglas de transición

- En los estados READ_LEN_HI, READ_LEN_LO, READ_CHECK_HI, READ_CHECK_LO: el byte debe ser un dígito hexadecimal en mayúsculas (0-9, A-F). Si no lo es, error.
- En EXPECT_LEN_SEPARATOR, EXPECT_CHECK_SEPARATOR: el byte debe ser :. Si no, error.
- En READ_BODY: se acumula el byte en body_buf, se incrementa body_pos. Cuando body_pos == body_len, se pasa a EXPECT_CHECK_SEPARATOR. Si el buffer se desborda (body_pos >= PROTOCOL_MAX_BODY_SIZE), error.
- En EXPECT_END: se recibe \n. Se valida el checksum con protocol_validate() de la etapa 1, se verifica que el cuerpo tenga la forma TTT:PAYLOAD (al menos 5 caracteres, con un : en la posición 3). Si todo es correcto, se devuelve PARSER_RESULT_MESSAGE_READY. Si no, PARSER_RESULT_ERROR.

7.2.3. 2.3 Recuperación ante error

Ante cualquier error (byte inesperado, checksum inválido, overflow, formato inválido), el parser debe:

1. Resetear todos los campos internos.
2. Volver al estado WAIT_START.
3. Si el byte que causó el error es @, reutilizarlo como inicio de una trama nueva (transicionar a READ_LEN_HI).

Esta regla de reutilización de @ es clave para no perder tramas válidas cuando hay ruido en la línea. El apunte 13 explica el mecanismo con un ejemplo paso a paso en la sección “Resincronización”.

7.2.4. 2.4 Integración con la cola de UART

El esqueleto ya tiene una tarea de FreeRTOS que recibe bytes de la cola uart_rx_queue. Tu parser se integra ahí:

```

void vUartRxTask(void *pvParameters) {
    parser_t parser;
    parser_init(&parser);

    protocol_message_t msg;

    for (;;) {
        uint8_t byte;
        if (xQueueReceive(uart_rx_queue, &byte, portMAX_DELAY) != pdPASS) {
            continue;
        }

        parser_result_t result = parser_consume_byte(&parser, byte, &msg);

        if (result == PARSER_RESULT_MESSAGE_READY) {
            app_handle_message(&msg);
        }
    }
}

```

```

    }
    // IN_PROGRESS y ERROR no requieren acción especial
}
}

```

7.2.5. 2.5 Despacho de comandos

La función `app_handle_message()` recibe el mensaje validado y decide qué hacer según el tipo y el payload:

```

void app_handle_message(const protocol_message_t *msg) {
    char response_buf[PROTOCOL_MAX_FRAME_LENGTH + 1];

    // Solo se procesan comandos de tipo CMD
    if (msg->type != PROTOCOL_TYPE_CMD) {
        protocol_encode("ERR", "code=unexpected_type",
                        response_buf, sizeof(response_buf));
        uart_tx_send(response_buf);
        return;
    }

    // Comandos reconocidos
    if (strcmp(msg->payload, "ping") == 0) {
        protocol_encode("ACK", "pong=1",
                        response_buf, sizeof(response_buf));
    } else if (strcmp(msg->payload, "led=on") == 0) {
        gpio_clear(GPIOC, GPIO13);
        protocol_encode("ACK", "cmd=ok",
                        response_buf, sizeof(response_buf));
    } else if (strcmp(msg->payload, "led=off") == 0) {
        gpio_set(GPIOC, GPIO13);
        protocol_encode("ACK", "cmd=ok",
                        response_buf, sizeof(response_buf));
    } else if (strcmp(msg->payload, "led=toggle") == 0) {
        gpio_toggle(GPIOC, GPIO13);
        protocol_encode("ACK", "cmd=ok",
                        response_buf, sizeof(response_buf));
    } else {
        // Comando no reconocido
        protocol_encode("ERR", "code=unknown_cmd",
                        response_buf, sizeof(response_buf));
    }

    uart_tx_send(response_buf);
}

```

La función `uart_tx_send()` ya existe en el esqueleto y transmite la trama completa por USART1. No tenés que implementarla.

El LED de la Blue Pill (PC13) enciende con nivel bajo: - `gpio_clear(GPIOC, GPIO13)` → LED encendido. - `gpio_set(GPIOC, GPIO13)` → LED apagado. - `gpio_toggle(GPIOC, GPIO13)` → conmuta.

La inicialización de PC13 como salida push-pull ya está hecha en el esqueleto (`firmware/app/actuators.c`).

7.3 Preguntas obligatorias de la etapa 2

1. ¿Por qué el parser debe ser incremental (byte a byte) en vez de leer una línea completa con un buffer y después parsear? Menciona al menos dos situaciones concretas donde una lectura por línea fallaría.
2. `\r` (0x0D) se ignora sin avanzar de estado. Si no se ignorara y se tratara como un byte normal, ¿en qué estado de la FSM causaría un error y por qué?
3. Dos tramas llegan pegadas por UART: `@08:CMD:ping:52\n@0A:CMD:led=on:6A\n`. Después de procesar la primera, ¿el parser entrega correctamente la segunda? Trazá los estados de la segunda trama y explicá por qué funciona sin necesidad de un delay entre tramas.

7.4 Evidencia mínima

- Código completo del enum de estados, la estructura `parser_t` y la función `parser_consume_byte()`.
- Código de `app_handle_message()` con los cinco comandos.
- Captura del monitor serie mostrando el envío de cada trama de comando y la trama de respuesta correspondiente (ver tabla de ejemplos en la consigna general). Para los comandos de LED, incluir foto del LED en el estado que corresponde.
- Al menos un caso donde el parser recibe basura antes de una trama válida (por ejemplo, `xyz@08:CMD:ping:52\n`) y aun así entrega el mensaje correcto.

8 Etapa 3: Estado, errores y resincronización

8.1 Consigna

Hasta ahora tu firmware responde a comandos conocidos con `ACK` y controla el LED. En esta etapa completás la lógica con tres piezas que faltan:

1. **Comando `status?`**: responde con una trama `STS` que reporta los contadores internos de tu firmware.
2. **Manejo de errores**: comandos no reconocidos y tipos de mensaje inesperados deben responderse con tramas `ERR` (ya lo implementaste en la etapa 2, acá lo verificás).
3. **Resincronización del parser**: asegurarte de que el parser se recupere correctamente después de ruido, bytes basura o tramas corruptas, sin perder tramas válidas que lleguen a continuación.

8.2 Subtareas

8.2.1. 3.1 El comando `status?`

Cuando tu firmware recibe `CMD:status?`, debe responder con una trama `STS` cuyo payload tenga este formato exacto:

`rx=N,ae=N,irq=N,pb=N,pm=N,pe=N,qd=N`

Cada campo es un contador que actualizás dentro de tu firmware:

Campo	Qué cuenta	Cuándo se incrementa
<code>rx</code>	Comandos procesados por la aplicación	Cada vez que <code>app_handle_message()</code> recibe un <code>CMD</code> conocido y lo procesa
<code>ae</code>	Errores de aplicación	Cada vez que se responde con <code>ERR</code>
<code>irq</code>	Bytes recibidos por la ISR de UART	El esqueleto ya cuenta esto: lee la variable <code>uart_rx_byte_count</code> en <code>firmware/app/tasks.c</code>
<code>pb</code>	Bytes procesados por el parser	Cada vez que llamás a <code>parser_consume_byte()</code>
<code>pm</code>	Mensajes válidos entregados	Cada vez que el parser devuelve <code>PARSER_RESULT_MESSAGE_READY</code>

Campo	Qué cuenta	Cuándo se incrementa
pe	Errores del parser	Cada vez que el parser devuelve <code>PARSER_RESULT_ERROR</code>
qd	Bytes descartados por cola llena	El esqueleto ya cuenta esto: lee <code>uart_rx_queue_drops</code>

Los contadores `irq` y `qd` ya existen en el esqueleto. Los demás (`rx`, `ae`, `pb`, `pm`, `pe`) tenés que agregarlos como variables estáticas en tu módulo de aplicación.

Para responder a `status?`, construí el payload con `snprintf()` y lo empaquetás con `protocol_encode()`:

```
char status_payload[PROTOCOL_MAX_PAYLOAD_LENGTH + 1];
snprintf(status_payload, sizeof(status_payload),
         "rx=%u,ae=%u,irq=%u,pb=%u,pm=%u,pe=%u,qd=%u",
         rx_count, ae_count, uart_rx_byte_count,
         pb_count, pm_count, pe_count, uart_rx_queue_drops);

protocol_encode("STS", status_payload, response_buf, sizeof(response_buf));
uart_tx_send(response_buf);
```

8.2.2. 3.2 Manejo de errores de aplicación

La función `app_handle_message()` de la etapa 2 ya contempla dos casos de error. Verificá que funcionen correctamente:

1. **Comando desconocido:** el payload no es `ping`, `led=on`, `led=off`, `led=toggle` ni `status?`. Respuesta: `ERR:code=unknown_cmd`.
2. **Tipo inesperado:** el tipo del mensaje no es `CMD` (por ejemplo, llega un `ACK` o un `DAT`). Respuesta: `ERR:code=unexpected_type`.

En ambos casos, incrementá el contador `ae`.

La trama de error para comando desconocido debe ser `@14:ERR:code=unknown_cmd:2D\n`. Verificá que tu `protocol_encode()` la genere correctamente.

Para tipo inesperado, el payload es `code=unexpected_type`. El checksum depende de la longitud exacta del cuerpo (`ERR:code=unexpected_type` = 24 caracteres, `LL` = "18"). Calculalo con tu función y verificá que el bridge ROS 2 la acepte.

8.2.3. 3.3 Resincronización del parser

Probá el parser con estas secuencias de entrada y verificá que se comporte como se espera:

Caso A: basura antes de una trama válida.

`xyz@08:CMD:ping:52\n`

El parser debe ignorar `x`, `y`, `z` en `WAIT_START`, detectar `@`, y entregar el mensaje `CMD:ping`.

Caso B: ruido con @ en el medio.

`@0@08:CMD:ping:52\n`

Secuencia esperada: 1. `@` inicia una trama (`WAIT_START` → `READ_LEN_HI`). 2. `0` se lee como primer dígito de longitud (`READ_LEN_HI` → `READ_LEN_LO`). 3. `@` aparece donde se esperaba un dígito hexadecimal → `PARSER_RESULT_ERROR`. Pero como el byte del error es `@`, se reutiliza: el parser vuelve a `READ_LEN_HI` con `@` como nuevo inicio. 4. `0, 8, :, CMD:ping, :, 5, 2, \n` → mensaje válido entregado.

Si tu parser no reutiliza `@` en el paso 3, la trama `@08:CMD:ping:52\n` se pierde. Revisá la sección “Resincronización” del apunte 13 para la lógica exacta.

Caso C: checksum corrupto.

@08:CMD:ping:53\n

El parser debe recorrer todos los estados, llegar a `EXPECT_END`, validar el checksum, detectar que `0x53 != 0x52`, devolver `PARSER_RESULT_ERROR` e incrementar `pe`. Después debe volver a `WAIT_START`.

Caso D: dos tramas válidas pegadas.

@08:CMD:ping:52\n@0A:CMD:led=on:6A\n

El parser debe entregar dos mensajes: `CMD:ping` y luego `CMD:led=on`. Entre una y otra no debe perder bytes ni quedarse en un estado incorrecto. Este caso ya debería funcionar si la etapa 2 está bien implementada. Verificalo.

8.3 Preguntas obligatorias de la etapa 3

1. ¿Por qué conviene reportar el estado con contadores internos (`rx`, `ae`, `pm`, `pe`) en vez de leer los registros del periférico USART? Menciona al menos dos razones. ¿Qué información obtenés de los contadores que no podrías obtener leyendo hardware?
2. En el caso B de resincronización (@008:CMD:ping:52\n), si el parser NO reutilizara el @ del error, ¿cuántos mensajes entregaría y cuáles? Trazá la secuencia completa de estados para justificar tu respuesta.
3. Si el bridge manda @0B:CMD:status?:13\n y recibe una trama STS con payload `rx=5,ae=2,...`, ¿qué tópico de ROS 2 publica el bridge y con qué contenido? ¿Por qué el tipo STS y no ACK?

8.4 Evidencia mínima

- Código de `app_handle_message()` con el comando `status?` y los contadores actualizados.
- Captura del monitor serie mostrando el intercambio completo para `status?`: envío de @0B:CMD:status?:13\n, recepción de @XX:STS:rx=N,ae=N,irq=N,pb=N,pm=N,pe=N,qd=N:YY\n con valores reales.
- Captura mostrando el envío de un comando inventado (payload cualquiera) y la respuesta @14:ERR:code=unknown_cmd:2D\n.
- Captura del monitor serie para el caso B de resincronización (@008:CMD:ping:52\n), mostrando que se entrega el mensaje `CMD:ping` a pesar del ruido.

9 Preguntas obligatorias de cierre

Además de las preguntas de cada etapa, responder estas preguntas integradoras:

1. **Checksum XOR vs. otros mecanismos:** El protocolo usa XOR byte a byte como checksum. ¿Qué tipo de errores detecta y cuáles no? Si en una trama se intercambian dos bytes del cuerpo sin modificar los demás, ¿el checksum XOR lo detecta? ¿Qué alternativa usarías si necesitaras mayor robustez sin cambiar la estructura de la trama?
2. **Longitud y temporización:** ¿Cuántos milisegundos tarda en transmitirse una trama de 64 caracteres (la máxima) a 115200 baud, 8N1? Si LL dice FF pero el cuerpo real mide 10 bytes, ¿qué hace el parser y cuánto tiempo queda esperando antes de detectar el error? ¿Por qué este escenario no debería ocurrir si el emisor respeta el protocolo?
3. **Resincronización completa:** Trazá estado por estado lo que ocurre cuando el parser recibe la secuencia ###08:CMD:ping:52\n. ¿Cuántos mensajes entrega? ¿En qué se diferencia del caso donde la basura contiene una arroba suelta (@008:CMD:ping:52\n, tratado en la etapa 3)?
4. **Conexión y niveles de tensión:** La Blue Pill funciona a 3.3 V. ¿Qué pasa si conectás PA9 y PA10 directamente a un adaptador USB-UART que trabaja a 5 V? ¿Es seguro? ¿Qué harías para proteger los pines del micro? ¿Afecta esto al protocolo o solamente al hardware?

10 Evidencia mínima exigida

La entrega no se considera completa si falta alguno de estos elementos:

- Código completo de `protocol_encode()`, `protocol_validate()`, `protocol_checksum()` y `hex_char_to_nibble()` (etapa 1).
- Código completo del parser: enum de estados, estructura `parser_t` y función `parser_consume_byte()` (etapa 2).
- Código de `app_handle_message()` con los cinco comandos y los contadores de estado (etapas 2 y 3).
- Capturas del monitor serie para las tres etapas, mostrando las tramas crudas (no solo el resultado):
 - Etapa 1: salida de `protocol_encode()` para al menos tres comandos, comparada con los valores de referencia.
 - Etapa 2: cada comando de LED enviado como trama y su respuesta ACK, con foto del LED.
 - Etapa 3: `status?` con contadores reales, comando desconocido con ERR, y caso de resincronización con ruido.
- Foto del conexionado entre Blue Pill y adaptador USB-UART.
- Respuestas a las 3 preguntas de cada etapa (9 en total).
- Respuestas a las 4 preguntas de cierre.

11 Evaluación

Puntaje total: **10 puntos**. La evaluación se distribuye en cuatro categorías:

- **Framing y checksum:** `protocol_encode()` genera tramas correctas para todos los comandos, `protocol_validate()` discrimina tramas válidas de inválidas, respuestas de la etapa 1 con ejemplos concretos.
- **Parser incremental:** FSM correcta con los 9 estados, consumo byte a byte desde la cola de UART, entrega mensajes completos con tipo y payload, ignora `\r`, resincroniza reutilizando @, respuestas de la etapa 2.
- **Comandos y respuestas:** `ping`, `led=on`, `led=off`, `led=toggle` y `status?` funcionan según la tabla de la consigna general, respuestas ACK y STS con formato y checksum correctos, comandos desconocidos devuelven ERR, respuestas de la etapa 3.
- **Diagnóstico y comprensión del protocolo:** respuestas a las 4 preguntas de cierre, razonamiento sobre checksum, temporización, resincronización y conexionado eléctrico, trazados de parser ante entradas con ruido.

Condición de aprobación sugerida: mínimo 6/10 y evidencia en las tres etapas.

12 Checklist de entrega

Antes de entregar, conviene verificar:

- ☐ `protocol_encode("CMD", "ping", ...)` produce `@08:CMD:ping:52\n`.
- ☐ `protocol_encode("CMD", "led=on", ...)` produce `@0A:CMD:led=on:6A\n`.
- ☐ `protocol_encode("CMD", "led=toggle", ...)` produce `@0E:CMD:led=toggle:7D\n`.
- ☐ `protocol_encode("ACK", "cmd=ok", ...)` produce `@0A:ACK:cmd=ok:6B\n`.
- ☐ `protocol_encode("ERR", "code=unknown_cmd", ...)` produce `@14:ERR:code=unknown_cmd:2D\n`.
- ☐ `protocol_validate()` devuelve 0 para tramas válidas y -1 para checksum alterado.
- ☐ El parser recorre los 9 estados correctamente y entrega `PARSER_RESULT_MESSAGE_READY` al recibir `\n` con checksum válido.
- ☐ `\r` (0x0D) se ignora; no produce error ni avance de estado.
- ☐ El parser se resincroniza tras ruido: descarta la trama parcial y reutiliza @ si el byte del error es @.
- ☐ `ping` responde `@0A:ACK:pong=1:22\n`.
- ☐ `led=on` enciende el LED (PC13 en bajo) y responde `@0A:ACK:cmd=ok:6B\n`.
- ☐ `led=off` apaga el LED (PC13 en alto) y responde `@0A:ACK:cmd=ok:6B\n`.

- ☐ `led=toggle` conmuta el LED y responde `@0A:ACK:cmd=ok:6B\n`.
- ☐ `status?` responde con trama STS y payload `rx=N,ae=N,irq=N,pb=N,pm=N,pe=N,qd=N`.
- ☐ Comando desconocido responde `@14:ERR:code=unknown_cmd:2D\n`.
- ☐ Mensaje con tipo distinto de CMD responde `ERR:code=unexpected_type`.
- ☐ El monitor serie está configurado a 115200 baud, 8N1, sin control de flujo.
- ☐ TX de la Blue Pill (PA9) va a RX del adaptador, RX de la Blue Pill (PA10) va a TX del adaptador.
- ☐ El GND de la Blue Pill y del adaptador están conectados entre sí.
- ☐ Las preguntas de cada etapa están respondidas con razonamiento técnico.
- ☐ Las 4 preguntas de cierre incluyen cálculos y trazados, no solo explicaciones cualitativas.